

Multi-Source E-Commerce Price Trend Analyzer

MINI PROJECT SYNOPSIS

Submitted in partial fulfilment of the requirement for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering – AIML

Submitted to

DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, UTTAR PRADESH, LUCKNOW



Submitted by

Stud Name: Praveen Kumar Singh

Roll No: 2501531530150

Section: 2F

Under the Supervision of

Guide Name: Bhawana Chaudhary

Assistant Professor



Department of Computer Science & Engineering
Lloyd Institute of Engineering & Technology
Plot No. 3, Knowledge Park II, Greater Noida, Uttar Pradesh 201306
Session: 2026–27

INDEX

Sr. No.	Topic	Page No.
01	Introduction	1 Page
02	Statement of Problem with Objective	2 Page
03	Literature Survey	3 Page
04	Expected Outcome / Scope of the Project	4 Page
05	Tentative Work Plan (mapped to the 10-week timeline)	5 Page
06	Software/Hardware required for the development of the project	6 Page
07	References	7 Page

Student's Signature

Guide's Signature

INTRODUCTION

Project Overview

The modern e-commerce landscape features severe price dispersion across multiple digital retail platforms, including Amazon, Flipkart, Croma, and direct-to-consumer storefronts. Dynamic pricing algorithms frequently alter product prices based on consumer demand patterns, stock availability, regional promotions, and competitor price tracking. Consequently, consumers struggle to identify true historical discount values and determine the optimal purchase timing.

The **Multi-Source E-Commerce Price Trend Analyzer** is a data engineering and analytical platform designed to track, unify, and analyze product price volatility across disparate online channels. The system resolves ambiguous product naming conventions across retailers, stores historical pricing time-series records in an embedded analytical columnar engine, and delivers interactive visual insights and price drop intelligence through a centralized web dashboard.

Domain & Project Classification

- **Project Domain:** Data Engineering, Natural Language Processing (Entity Resolution), and Applied Business Intelligence (OLAP Analytics).
- **Project Type:** Analytical Pipeline and Interactive Decision Support System.

Technology Stack & Architecture

- **Data Processing & Pipeline:** Python, `pandas`, and Regular Expressions for ingestion, structural schema transformation, and numerical cleansing.
- **Fuzzy String Matching & NLP:** `RapidFuzz` implementing Levenshtein distance metrics and Token Set Ratio heuristics for entity resolution.
- **Analytical Storage:** `DuckDB` embedded columnar database supporting structured SQL window aggregation functions.
- **Data Visualisation & Web Interface:** `Streamlit` with embedded dynamic `Plotly` multi-line charting.

STATEMENT OF PROBLEM AND OBJECTIVES

Problem Statement

Online retail platforms employ asymmetric naming conventions, differing attribute hierarchies, and uncoordinated promotional campaigns. For example, a single mobile phone may be catalogued as "*Apple iPhone 15 (128 GB) - Black*" on one platform and "*iPhone 15 Black 128GB Base Model*" on another.

Conventional price comparison systems face notable limitations:

- Manual comparisons across multiple browser tabs are tedious and prone to human error.
- Rule-based scrapers frequently break due to dynamic UI changes and aggressive anti-bot rate limits.
- Standard transactional databases (OLTP) exhibit high query latency when executing multi-day rolling average calculations across thousands of price change events.
- Consumers lack transparent visibility into whether an advertised "sale price" represents a genuine discount or an artificial markup from historical norms.

Objectives of the Proposed Project

1. **Automated Multi-Source Ingestion & Cleansing:** Develop automated Python pipeline modules to ingest heterogeneous product data feeds, strip non-numeric currency artifacts, and format timestamp fields consistently.
2. **Algorithmic Entity Matching:** Implement a fuzzy string matching module utilizing `RapidFuzz` (Token Set Ratio) to automatically map non-standard product titles to canonical inventory identifiers with a similarity threshold exceeding 65%.
3. **High-Performance Columnar Time-Series Storage:** Integrate `DuckDB` as an embedded OLAP storage engine to store structured historical pricing logs and execute time-series window functions.
4. **Historical Price Trend Analytics:** Formulate analytical SQL queries to compute 7-day rolling price averages, identify historical all-time lows and all-time highs, and detect cross-store price arbitrage margins.
5. **Interactive Analytical Dashboard:** Construct a responsive, web-based dashboard using `Streamlit` and `Plotly` providing interactive time-series trend lines, deal metric scorecards, and data drill-down tables.

LITERATURE SURVEY

The assessment of multi-source price tracking and entity matching spans e-commerce economics, record linkage algorithms, and analytical data architectures

Author(s) / Reference	Methodology / System Explored	Key Findings & Observations	Relevance to Proposed Project
Venkatesan et al. (2021)	Rule-based web scrapers using BeautifulSoup for real-time retail price monitoring.	Web scrapers suffer high fragility when retail platforms update front-end DOM elements, necessitating decoupling of ingestion from analysis.	Highlights the necessity of using structured intermediate data feeds and robust pipeline modularity
Christen, P. (2012)	Probabilistic and distance-based Record Linkage techniques for data integration.	String-distance heuristics (Levenshtein, Jaro-Winkler) significantly outperform strict keyword lookups on messy product catalogues.	Directly supports utilizing RapidFuzz Token Set Ratio heuristics for cross-platform entity matching.
Raasveldt & Mühleisen (2019)	In-process analytical database design (DuckDB) for vectorized data analysis.	Embedded columnar engines execute aggregations and rolling window functions orders of magnitude faster than relational OLTP engines like SQLite.	Governs the choice of DuckDB for historical time-series pricing data rollups without database server overhead.
Gorakala & Uzunalioglu (2020)	Time-series decomposition and rolling window algorithms in consumer retail trends.	Moving window averages (e.g., 7-day rolling means) effectively smooth out artificial promotional spikes to expose genuine baseline trends.	Direct basis for the analytical SQL queries calculating 7-day moving averages and deal scores.

EXPECTED OUTCOME AND SCOPE OF THE PROJECT

Expected Outcomes

- **Operational Data Pipeline:** A fully functional Python pipeline capable of standardising, matching, and storing multi-channel pricing records.
- **Accurate Entity Resolution Engine:** Automated matching of differently titled products from multiple store channels to single canonical entities with high precision.
- **Analytical Storage Layer:** A local `DuckDB` database file capable of sub-second execution of window aggregation queries on chronological pricing tables.
- **Interactive Visual Dashboard:** A web-based `Streamlit` portal featuring:
 - Dropdown selector for canonical products.
 - Real-time metric cards showing Best Current Store, Current Best Price, Historic Low, and 30-Day Mean.
 - Interactive `Plotly` multi-line comparison charts displaying price movements across competing retailers.
 - Raw record feed tab displaying matched cross-store records.

Target Users & Practical Applications

- **Consumer Decision Support:** Enables shoppers to make data-backed purchasing decisions based on actual historical discounts.
- **Retail Competitor Intelligence:** Provides small retailers with visibility into competitor price fluctuations and market discount timings.

Scope for Future Improvements

- **Automated Notification Daemons:** Integrating Discord or Telegram webhook triggers when prices drop below a specified threshold.
- **Predictive Forecasting Models:** Training lightweight linear regression or ARIMA/Prophet models to predict upcoming festive price drops.
- **Extended Source Expansion:** Incorporating real-time merchant APIs (such as eBay Developer API) and affiliate product data feeds.

TENTATIVE WORK PLAN

The project follows a phased 10-week lifecycle aligned with academic requirements:

Week	Phase / Milestone	Detailed Activity Description
Week 01	Problem Formulation & Scoping	Finalizing project domain, requirements elicitation, defining canonical product sets, and preparing synopsis documentation.
Week 02	Literature Survey & Design	Studying fuzzy string matching algorithms, evaluating database engines (OLTP vs. OLAP), and designing data pipeline architecture.
Week 03	Data Ingestion Pipeline Setup	Constructing Python ingestion modules, normalising timestamps, and cleaning currency and pricing symbols via regular expressions.
Week 04	Entity Resolution Implementation	Implementing RapidFuzz Token Set Ratio matching logic and evaluating matching accuracy against varying similarity thresholds.
Week 05	Database Architecture Integration	Setting up the embedded DuckDB database schema, ingesting processed records, and verifying data integrity.
Week 06	Analytical Query Development	Writing SQL analytical queries for 7-day rolling averages, historical minimum/maximum price tracking, and store spread metrics.
Week 07	Dashboard UI Development	Developing the frontend interface using <code>Streamlit</code> , integrating user selection dropdowns, and formatting metric cards.
Week 08	Dynamic Data Visualisation	Integrating Plotly interactive time-series charts, setting hover tooltips, and building tabular raw-feed audit views.
Week 09	System Integration & Testing	Performing end-to-end integration testing, verifying entity resolution edge cases, and optimising SQL query latency.
Week 10	Documentation & Presentation	Completing final project report, user guide, codebase documentation, and preparing the final defense presentation.

HARDWARE AND SOFTWARE REQUIREMENTS

Software Requirements

- **Operating System:** macOS (Apple Silicon)
- **Programming Language:** Python 3.9+
- **Development Environment (IDE):** Visual Studio Code or PyCharm
- **Analytical Database Engine:** DuckDB (v0.9+ embedded OLAP database)
- **Core Libraries & Frameworks:**
 - `pandas` (Dataframe manipulation and cleaning)
 - `rapidfuzz` (Fast C++ accelerated fuzzy string matching)
 - `duckdb` (In-process analytical SQL processing)
 - `streamlit` (Web application framework)
 - `plotly` (Interactive charting library)

Hardware Requirements

- **Processor:** Modern Dual-Core / Multi-Core Processor (Apple Silicon M-series)
- **Random Access Memory (RAM):** 4 GB minimum (8 GB recommended for in-memory analytical operations)
- **Storage:** 500 MB of available local storage for code, dependencies, and analytical database files
- **Display Resolution:** 1366×768 minimum display output for dashboard rendering

REFERENCES

1. Christen, P., *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*, Berlin: Springer-Verlag, 2012.
2. DuckDB Foundation, "DuckDB: An In-Process SQL OLAP Database Management System Documentation," 2023. Website URL: <https://duckdb.org/docs/>
3. McKinney, W., *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter*, 3rd ed., Sebastopol: O'Reilly Media, 2022.
4. Raasveldt, M., and Mühleisen, H., "DuckDB: an Embeddable Analytical Database," *Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data*, pp. 1981–1984, 2019.
5. RapidFuzz Project, "RapidFuzz: Accelerated String Matching for Python," 2023. Website URL: <https://github.com/maxbachmann/RapidFuzz>
6. Streamlit Inc., "Streamlit Application Framework Documentation," 2024. Website URL: <https://docs.streamlit.io/>
7. Venkatesan, R., Farris, P., and Wilcox, R. T., "Retail Dynamic Pricing and Promotional Analytics," *Journal of Interactive Marketing*, Vol. 48, pp. 112–127, 2021.